Deep Learning Lab 11-2 Visualization & Style Transfer

Datalab

Department of Computer Science,

National Tsing Hua University,

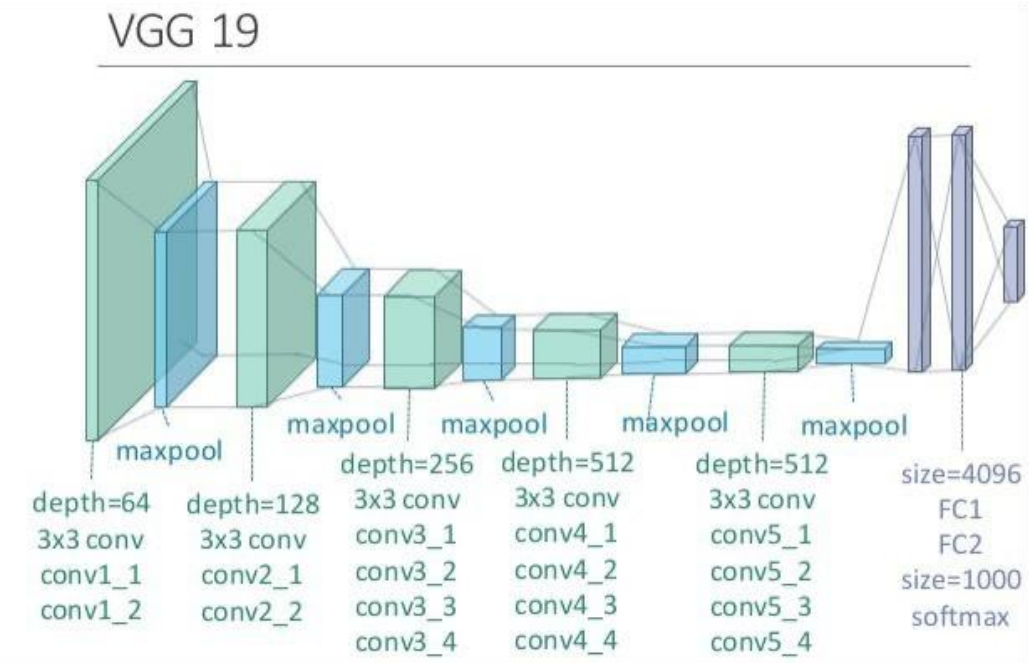
Taiwan

Outline

- Visualization
- Neural Style Transfer
  - A Neural Algorithm of Artistic Style
  - AdaIN (Adaptive Instance Normalization)

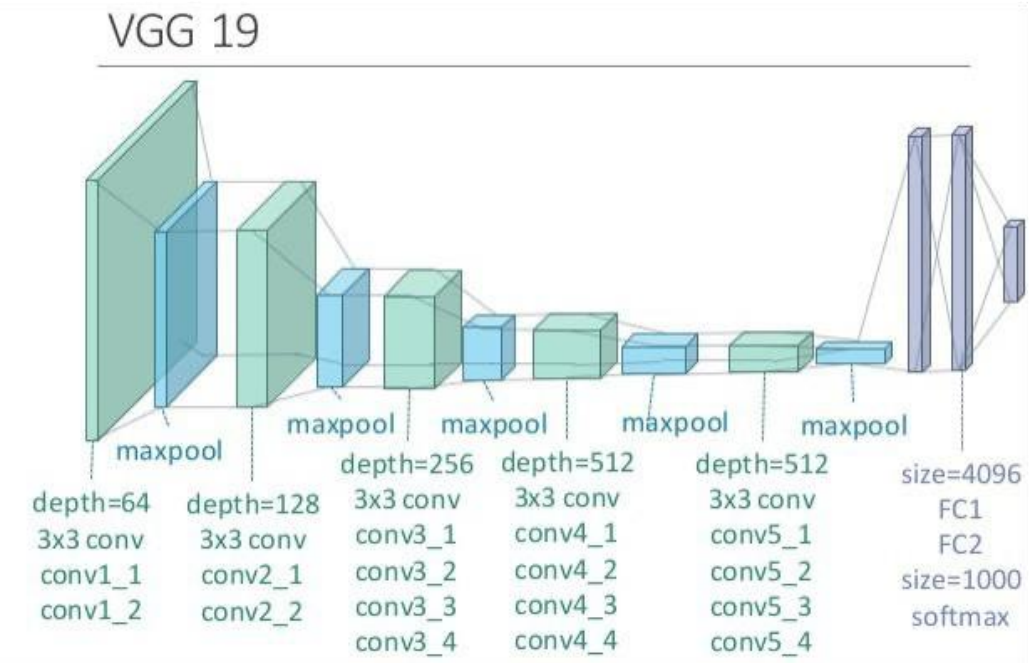
VGG19

- VGG19 is known for its simplicity, using only 3×3 convolutional layers stacked on top of each other in increasing depth.



VGG19

- CNN learns how to create useful representations of the data to differentiate between different classes.



VGG19

- In this tutorial, we are going to use VGG19 pretrained on ImageNet for visualization.
- ImageNet is a large dataset used in ImageNet Large Scale Visual Recognition Challenge(ILSVRC). The training dataset contains around 1.2 million images composed of 1000 different types of objects



More about Pretrained Networks...

- Pretrained network is useful and convenient for several further usages, such as style transfer, transfer learning, fine-tuning, and so on.
- Generally, using pretrained network can save a lot of time and also easier to train a model on more complex dataset or small dataset.

Outline

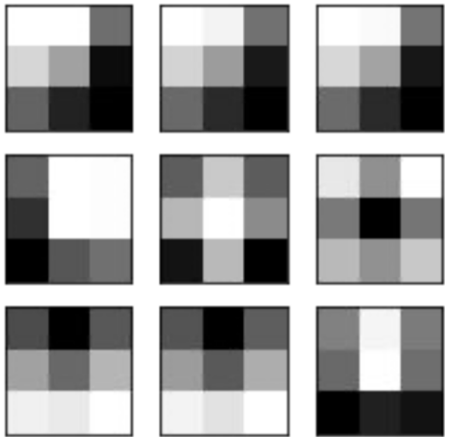
- Visualization
- Neural Style Transfer
  - A Neural Algorithm of Artistic Style
  - AdaIN (Adaptive Instance Normalization)
- Save and Load models (Checkpoints)

Visualize Filters

- Visualize the weights of the convolution filters to help us understand what neural network have learned.
- Learned filters are simply weights, yet because of the specialized two-dimensional structure of the filters, the weight values have a spatial relationship to each other and plotting each filter as a two-dimensional image is meaningful (or could be)

Visualize Filters

- Let's look at some filters in the first convolutional layer.
- The dark squares indicate small or inhibitory weights and the light squares represent large or excitatory weights.

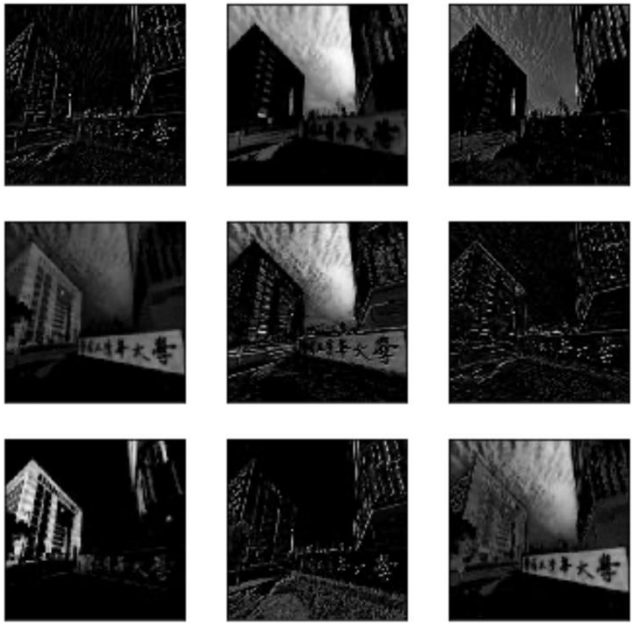


Visualize Feature Maps

- Activation maps, called feature maps, capture the result of applying the filters to input, such as the input image or another feature map.
- The idea of visualizing a feature map for a specific input image would be to understand what features of the input are detected or preserved in the feature maps

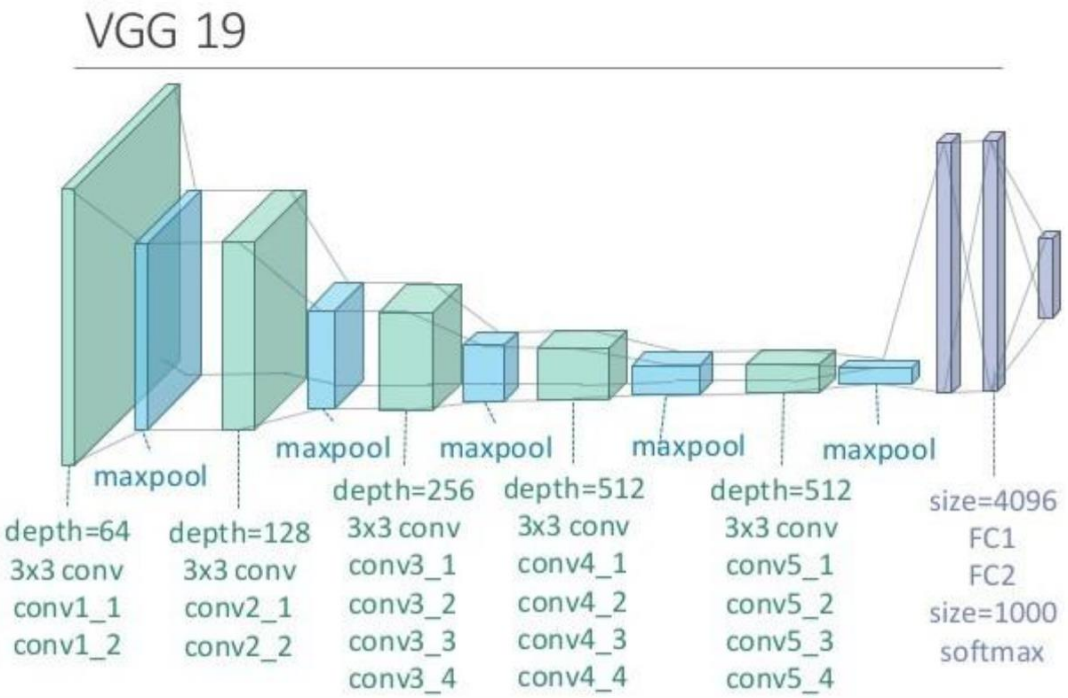
Visualize Feature Maps

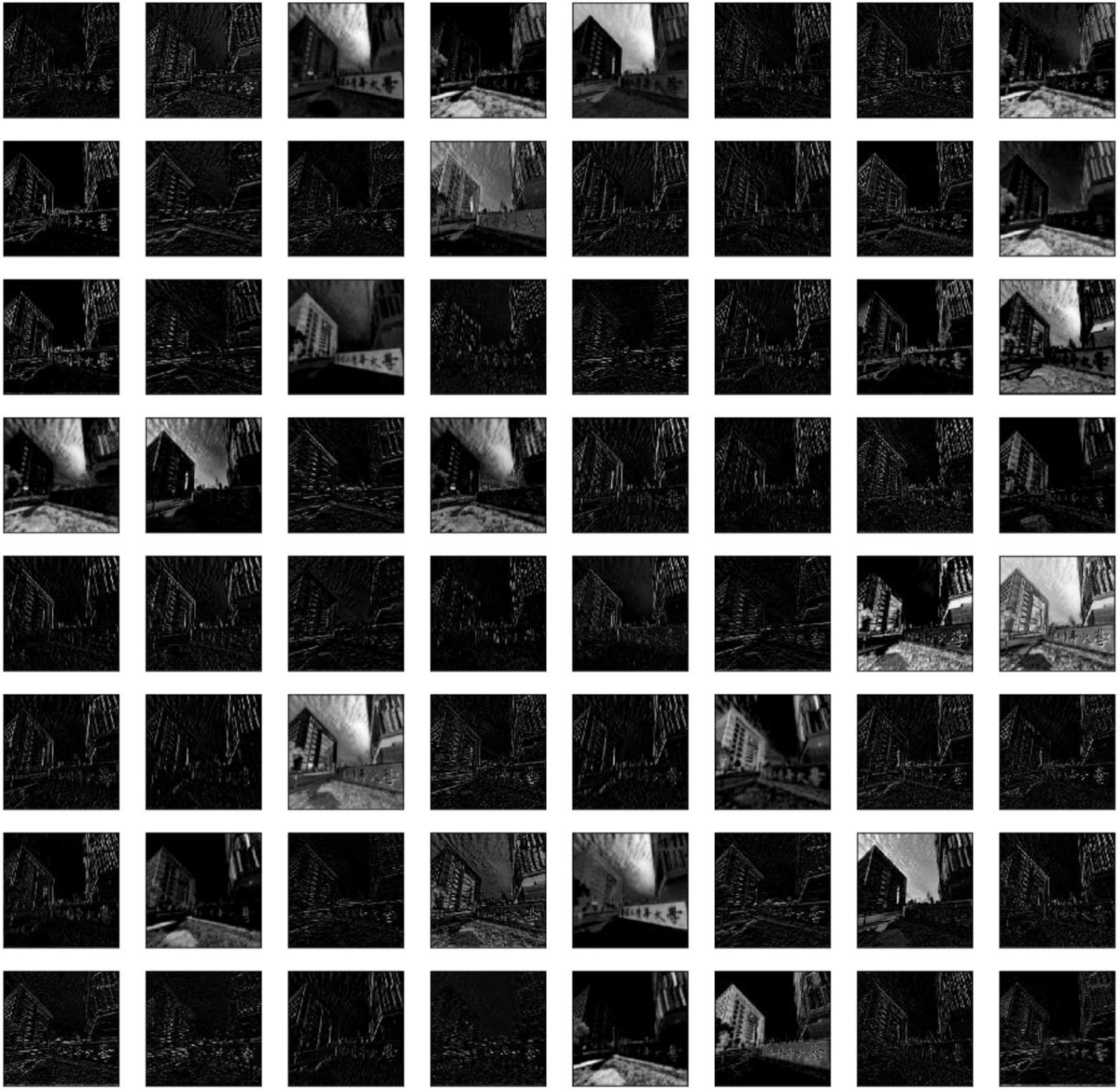
- Result of applying the filters in the first convolutional layer is a lot of versions of the input image with different features highlighted.

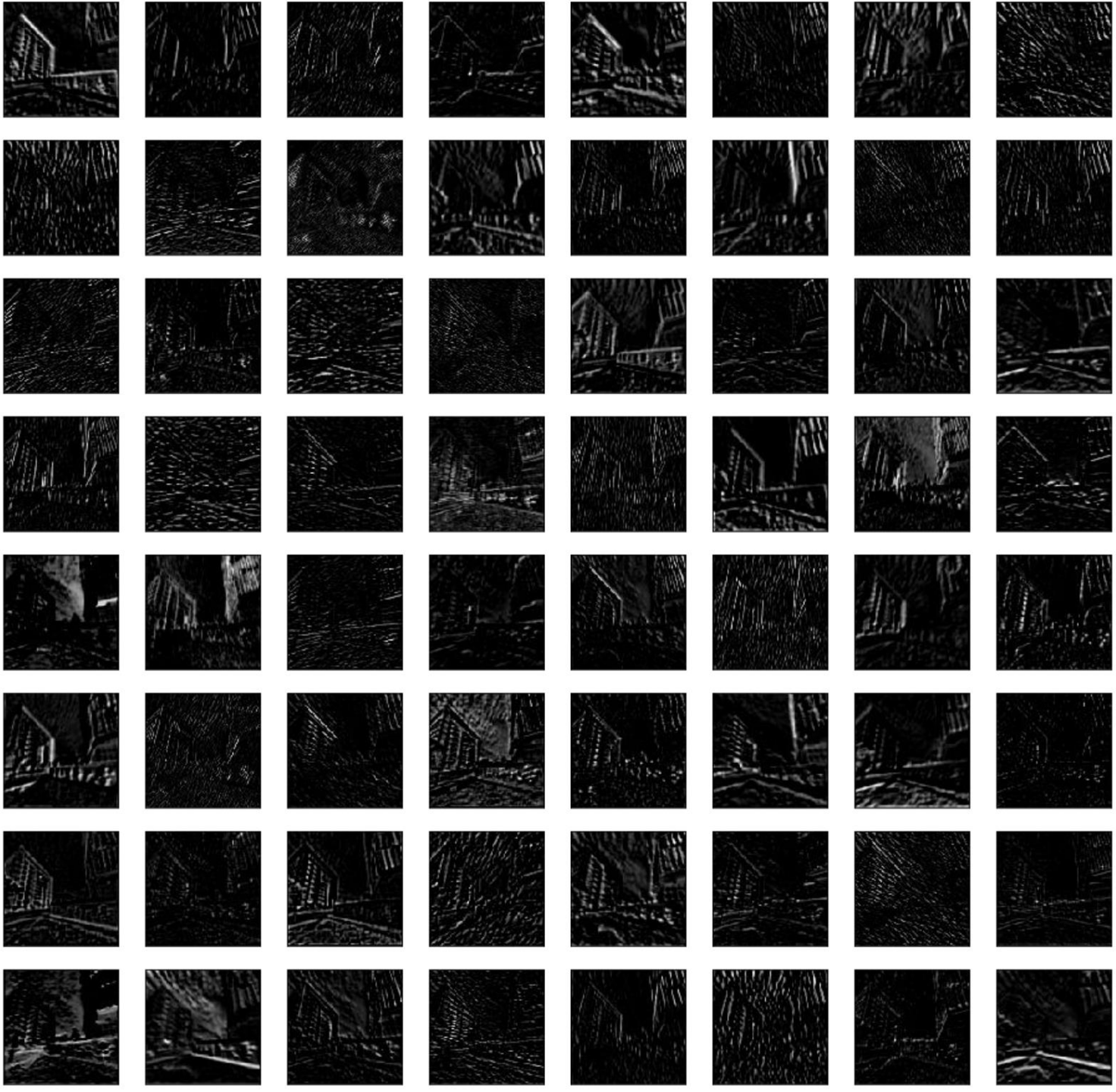


Visualize “Deeper” Feature Maps

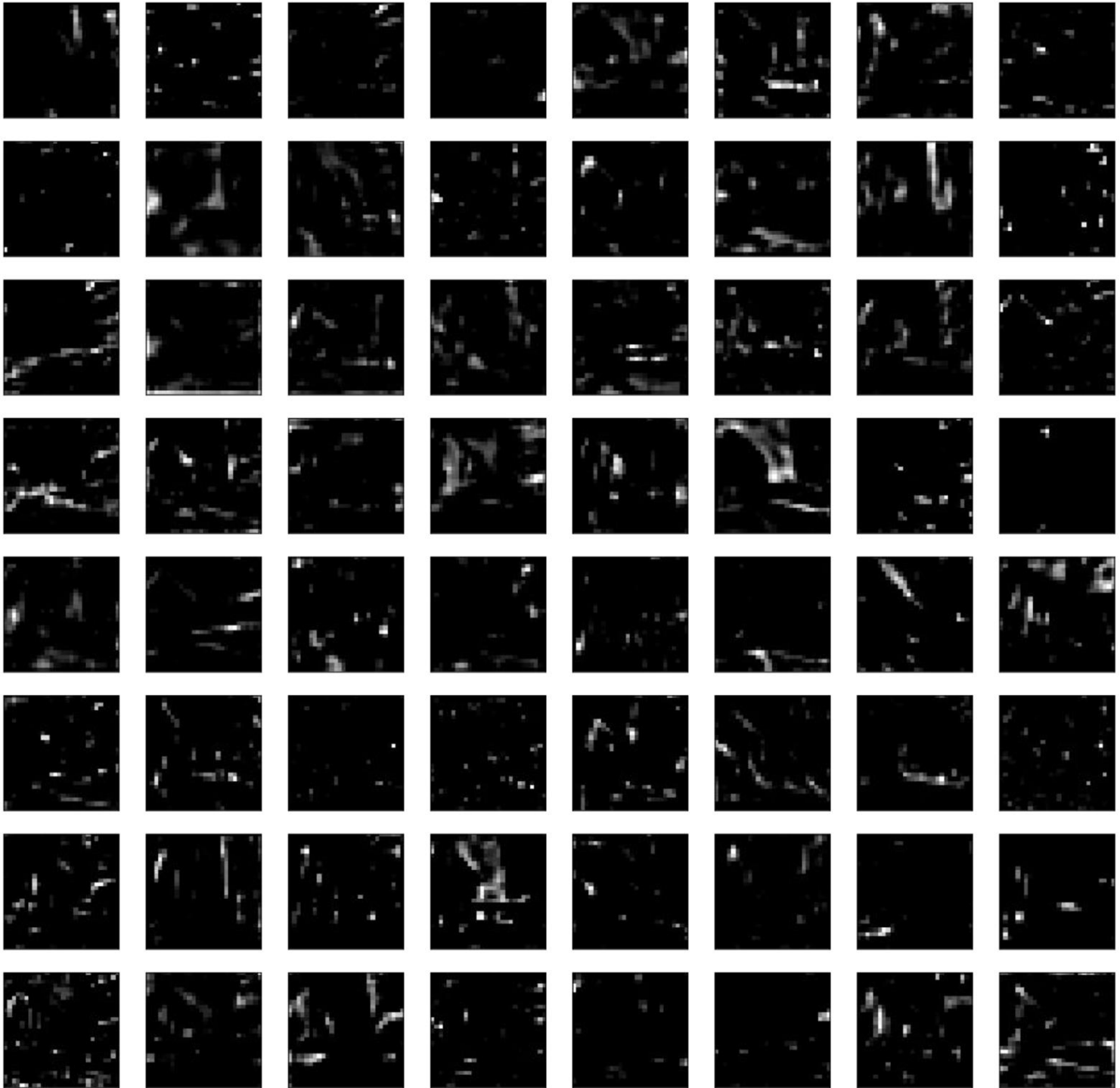
- Let's visualize the feature maps output from each block of VGG19.

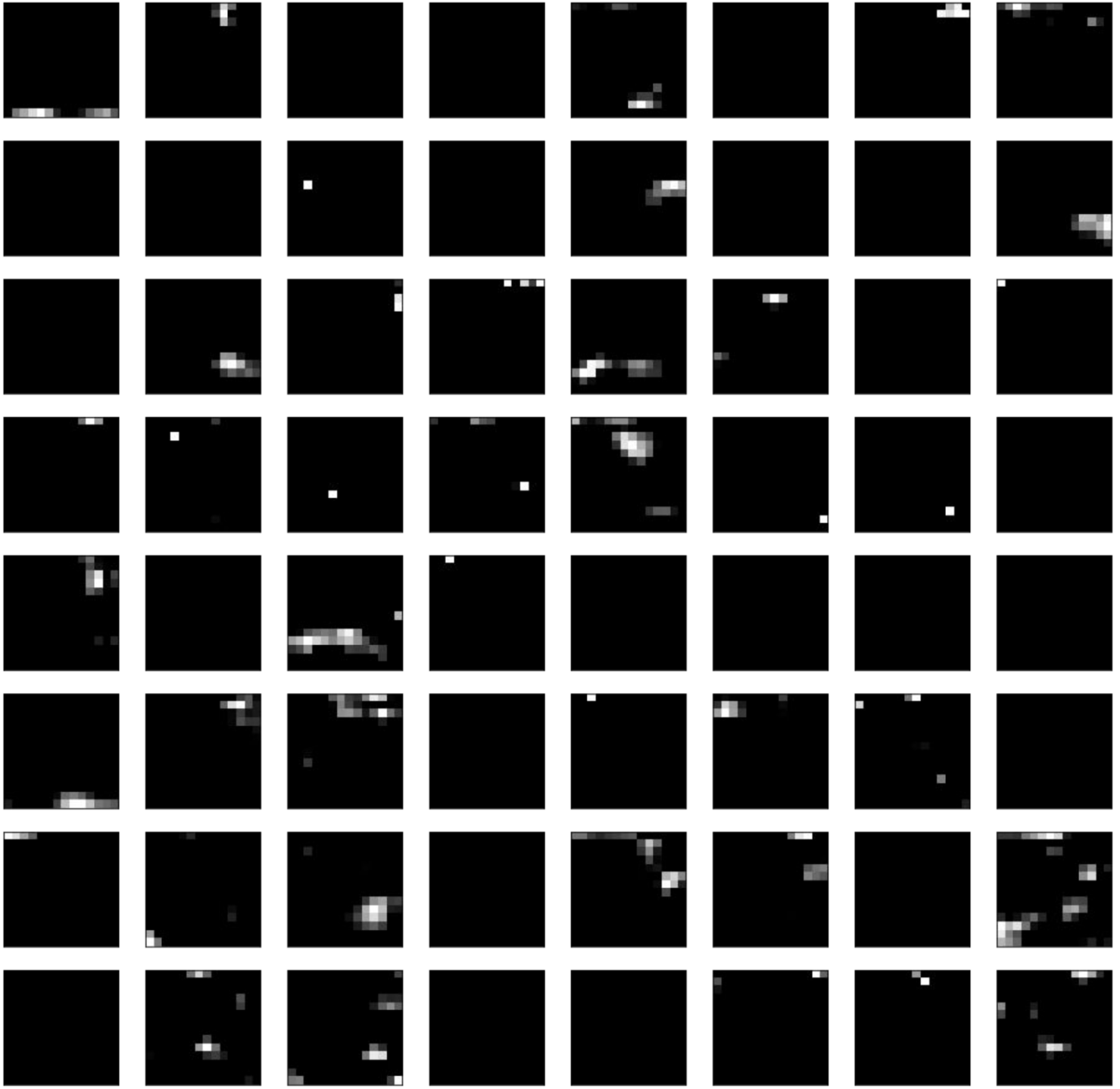










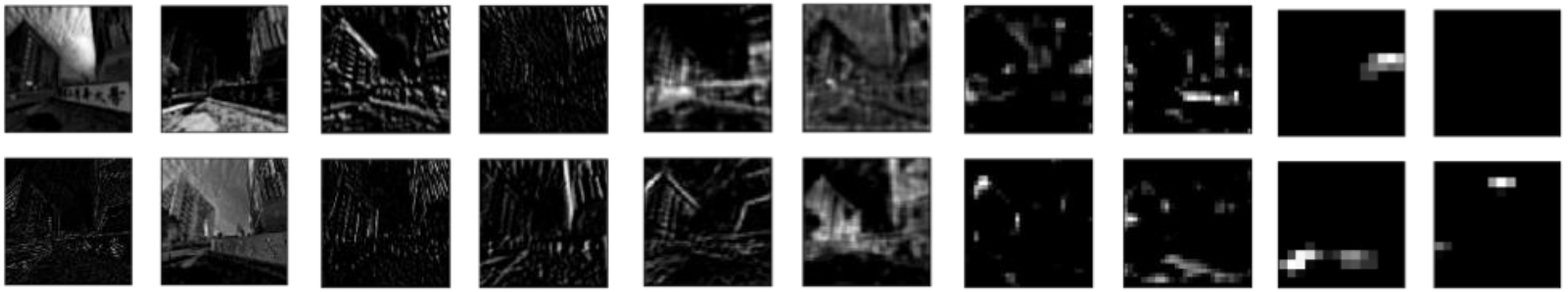


Visualize “Deeper” Feature Maps

- Feature maps closer to the input of the model capture a lot of fine detail in the image and that as we progress deeper into the model, the feature maps show less and less detail.
- This pattern was to be expected, as the model abstracts the features from the image into more general concepts that can be used to make a classification.
- block 1 block 2 block 3 block 4 block 5

Visualize “Deeper” Feature Maps

- block 1 block 2 block 3 block 4 block 5



Visualize Gradients

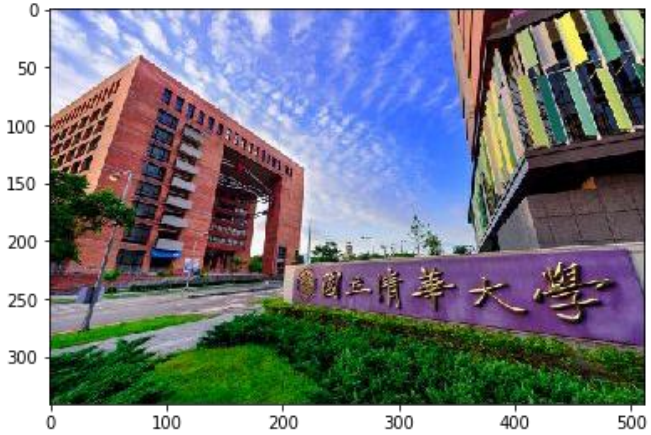
- Visualizing convolutional output is a pretty useful technique for visualizing shallow convolution layers.
- Unfortunately, when we get into the deeper layers, it’s hard to understand them just by just looking at the convolution output.

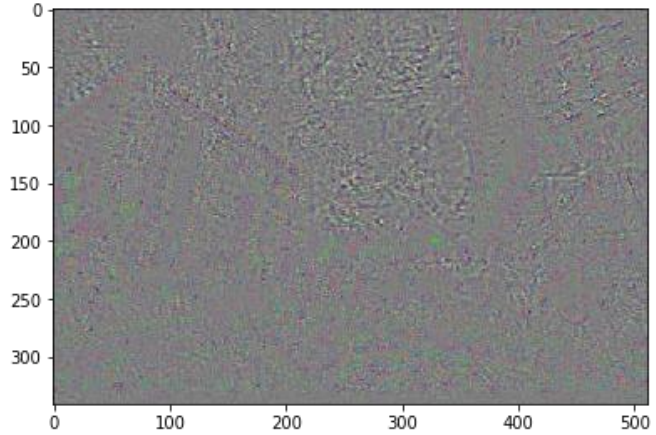
Visualize Gradients

- If we want to understand what the deeper layers are really doing, we can try to use backpropagation to show us the gradients of a particular neuron with respect to our input image, which is called saliency map.
- We will make a forward pass up to the layer that we are interested in, and then backpropagate to help us understand which pixels contributed the most to the activation of that layer.

Visualize Gradients

- Compute the gradient of maximum neurons among all activations in the required layer with respect to the input image.

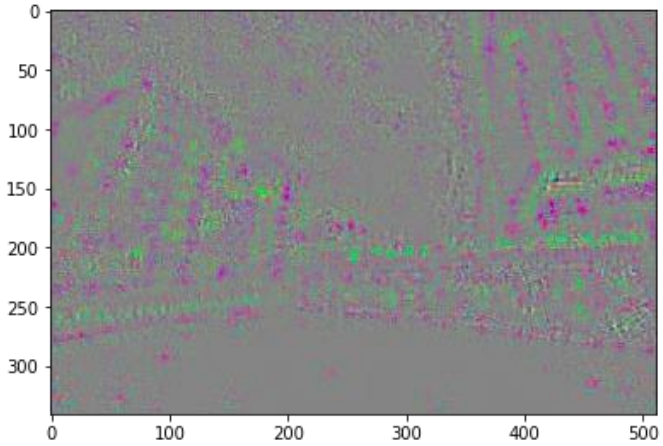




Visualize Gradients

- We can also visualize the gradient of any single feature map.



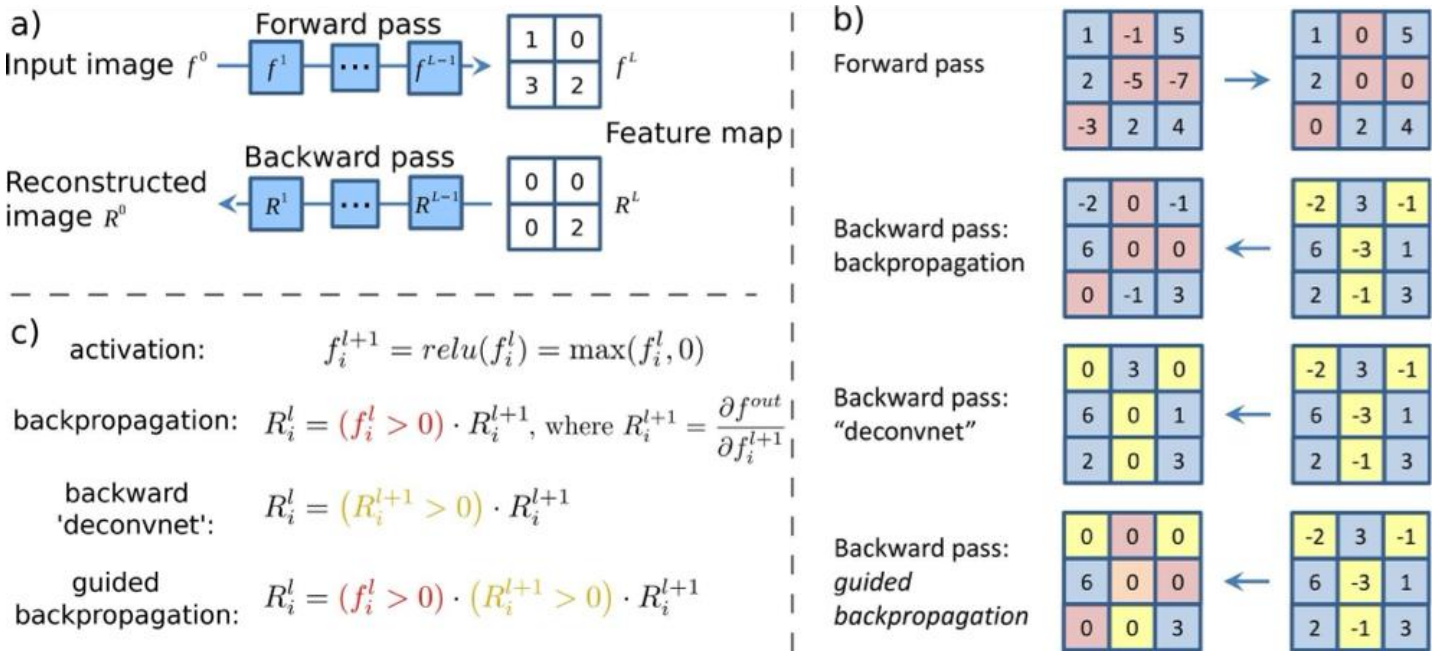


Guided-Backpropagation

- As we can see above, the results are still hard to explain and not very satisfying.
- Ideally, neurons act like detectors of particular image features. We are only interested in what image features the neuron detects, not in what kind of stuff it doesn't detect.
- Therefore, when propagating the gradient, we set all the negative gradients to 0.

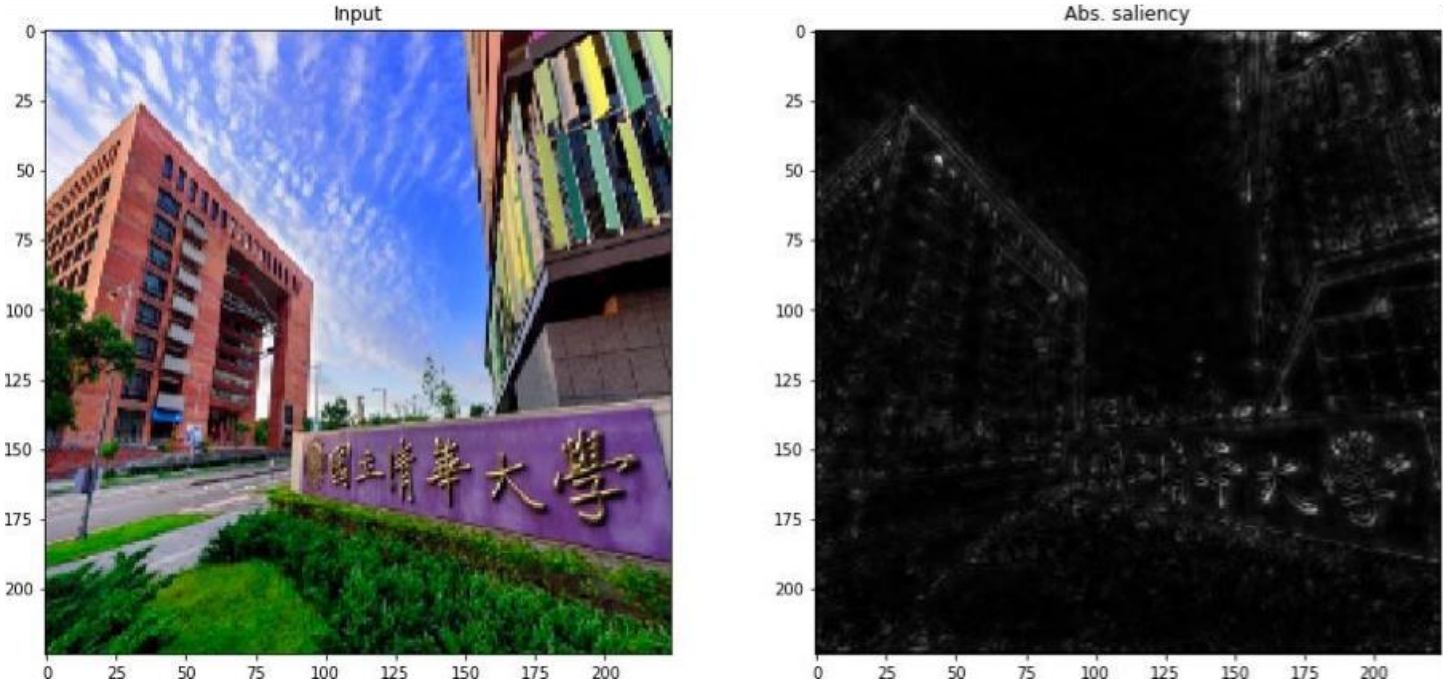
Guided-Backpropagation

- Thus, the gradient is "guided" by both the input and the error signal.



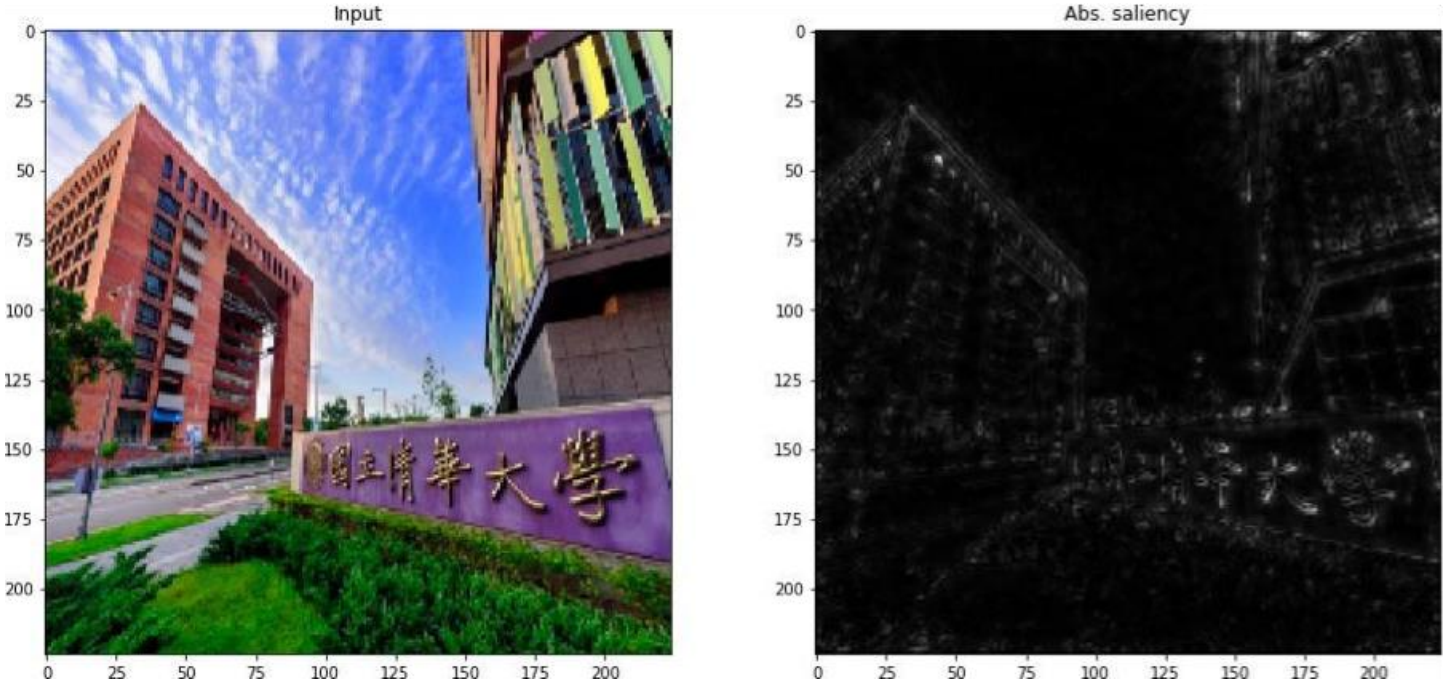
Guided-Backpropagation

- Thus, the gradient is “guided” by both the input and the error signal.



Guided-Backpropagation

- Thus, the gradient is “guided” by both the input and the error signal.



Outline

- Visualization
- Neural Style Transfer
  - A Neural Algorithm of Artistic Style
  - AdaIN (Adaptive Instance Normalization)

Neural Style Transfer

- Leon Gatys and his co-authors has a very interesting work called "A Neural Algorithm of Artistic Style" that uses neural representations to separate and recombine content and style of arbitrary images, providing a neural algorithm for the creation of artistic images





Neural Style Transfer







Outline

- Visualization
- Neural Style Transfer
  - A Neural Algorithm of Artistic Style
  - AdaIN (Adaptive Instance Normalization)

A Neural Algorithm of Artistic Style

- At a high level, in order for a network to perform image classification, it must understand the image.
- This requires taking the raw image as input pixels and building an internal representation that converts the raw image pixels into a complex understanding of the features present within the image.

A Neural Algorithm of Artistic Style

- Use the intermediate layers of the model to get the content and style representations of the image.
- Content representation
- Style representation

A Neural Algorithm of Artistic Style

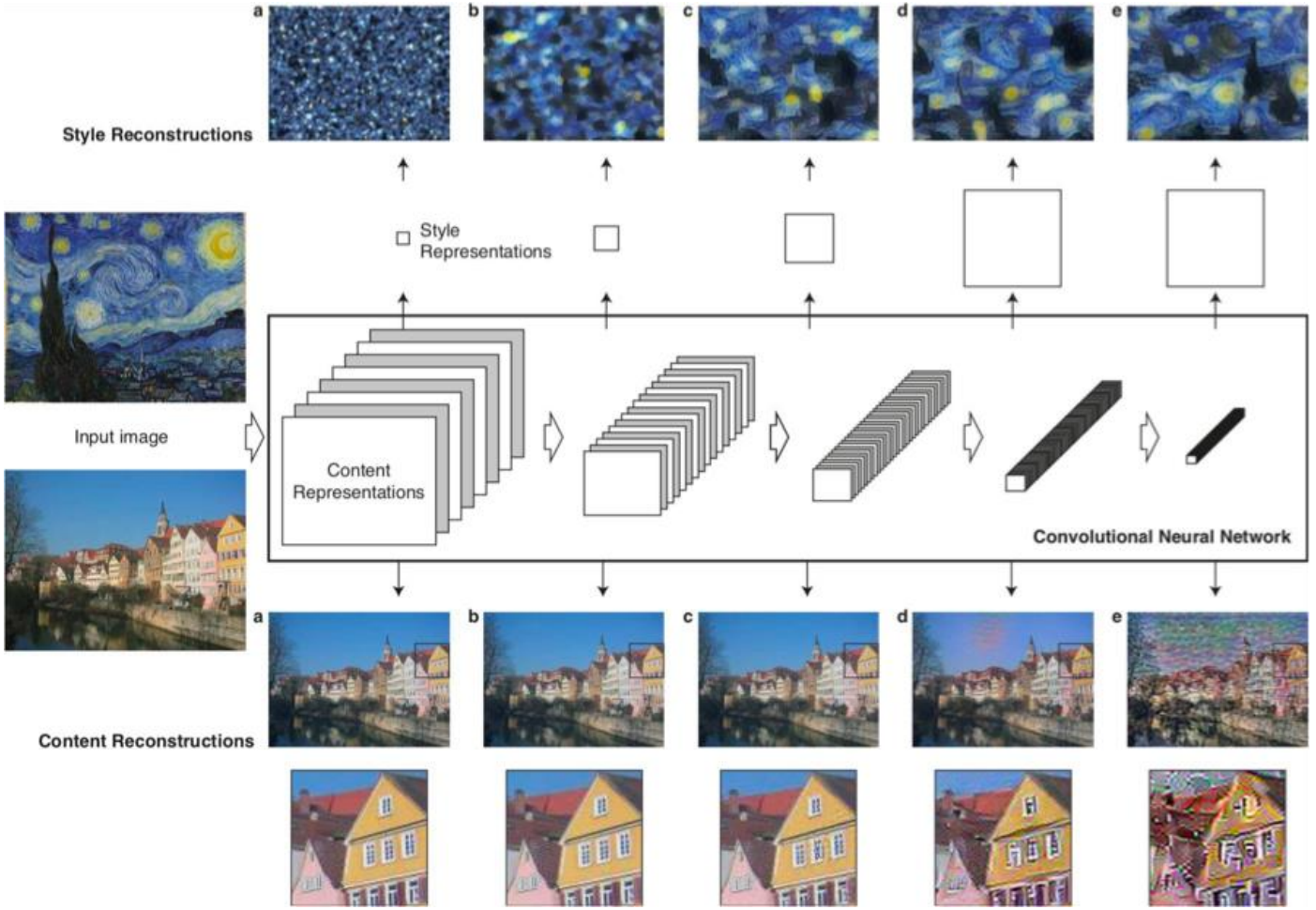
- Use the intermediate layers of the model to get the content and style representations of the image.
- Content representation
  - Content features of the content image is calculated by feeding the content image into the neural network, and extract the activations of those.

    `content_layers`
- Style representation

A Neural Algorithm of Artistic Style

- Use the intermediate layers of the model to get the content and style representations of the image.
- Content representation
- Style representation
  - For style features, we extract the correlation of the features of the style-image layer-wise (gram matrix). By adding up the feature correlations of multiple layers, which corresponding to style_layers, we obtain a multi-scale representation of the input image, which captures its texture information instead of the object arrangement in the input image

A Neural Algorithm of Artistic Style



A Neural Algorithm of Artistic Style

- Our goal is to create an output image which is synthesized by finding an image that simultaneously matches the content features of the photograph and the style features of the respective piece of art.
- We can define the loss function as the composition of the dissimilarity of:
  - The content features between the output image and the content image.
  - The style features between the output image and the style image .

A Neural Algorithm of Artistic Style

- Content loss
  - Minimize squared-error loss between two feature representations, where F and P are the representations of original and the generated image respectively.

$$\mathcal{L}_{\text{content}}(\vec{p}, \vec{x}, l) = \frac{1}{2} \sum_{i,j} \left(F_{ij}^{l} - P_{ij}^{l}\right)^2$$

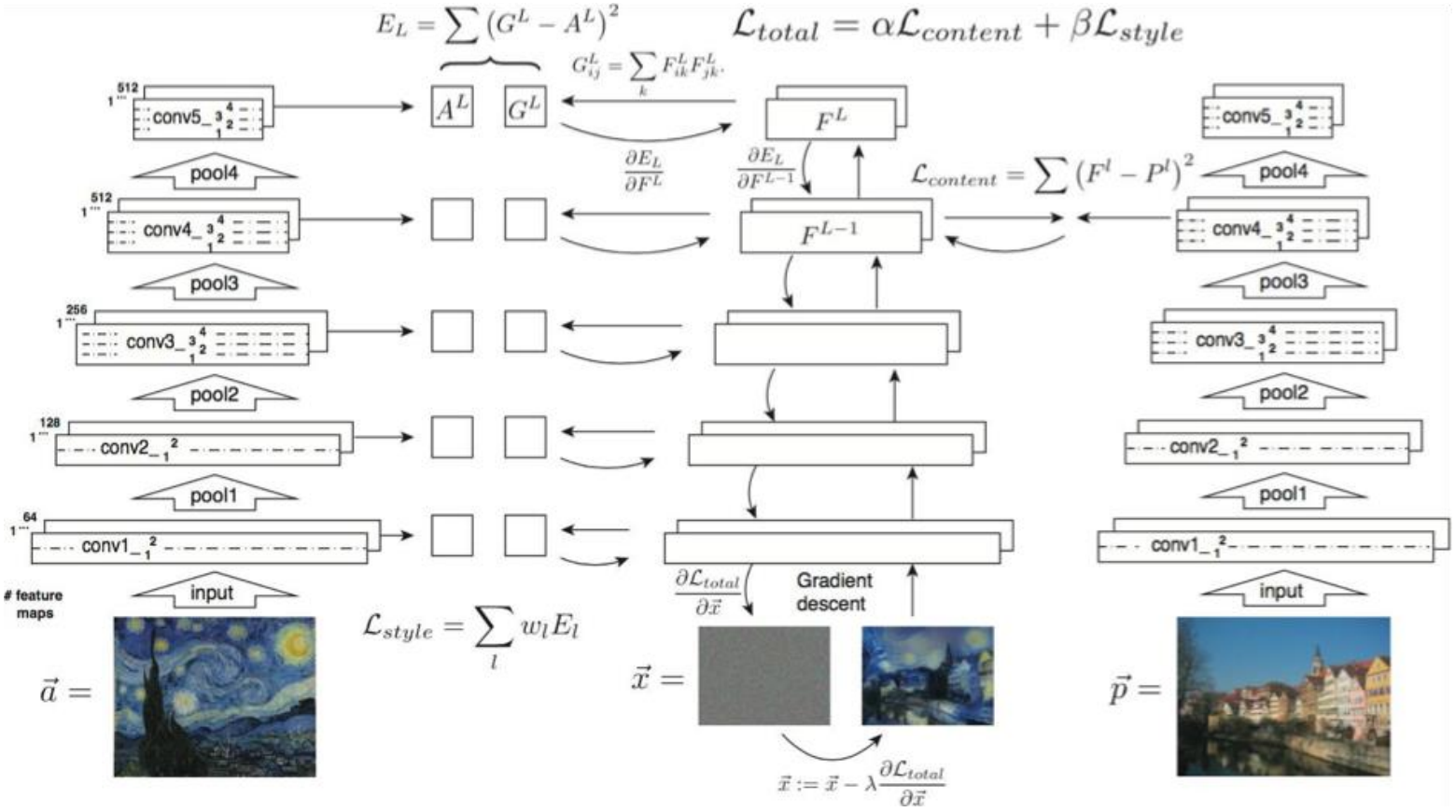
A Neural Algorithm of Artistic Style

- Style loss
  - Use Gram matrix to calculate style loss, consisting feature correlations between the different filter responses. Minimize the mean-squared distance between two Gram matrices. A and G represents original and generated image respectively, and w are weighting factors

$$G_{ij}^l = \sum_k F_{ik}^l F_{jk}^l \qquad E_l = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} \left(G_{ij}^l - A_{ij}^l\right)^2$$

$$\mathcal{L}_{\text{style}}(\vec{a}, \vec{x}) = \sum_{l=0}^{L} w_l E_l$$

A Neural Algorithm of Artistic Style



A Neural Algorithm of Artistic Style

- So far, we have learned how to train a neural network by using gradient descent to update the weights.

$$\frac{\partial L}{\partial W}$$

- How to use the loss function we have defined?
  - We have to update our input image!

$$\frac{\partial L}{\partial x}$$

A Neural Algorithm of Artistic Style

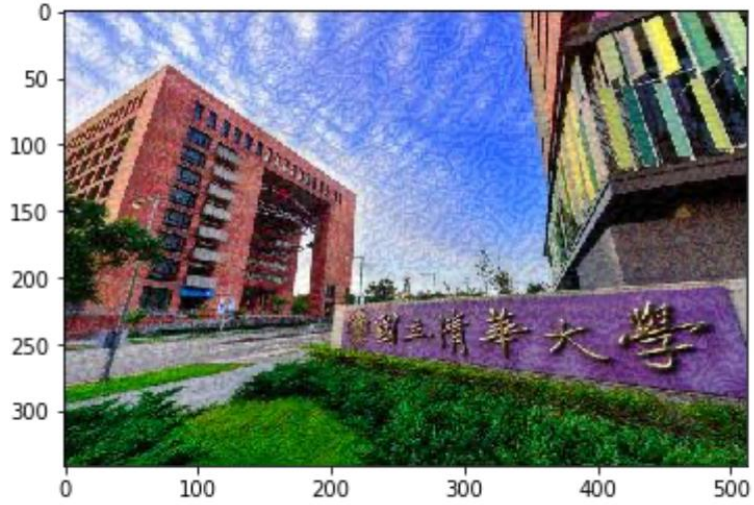
Content Image

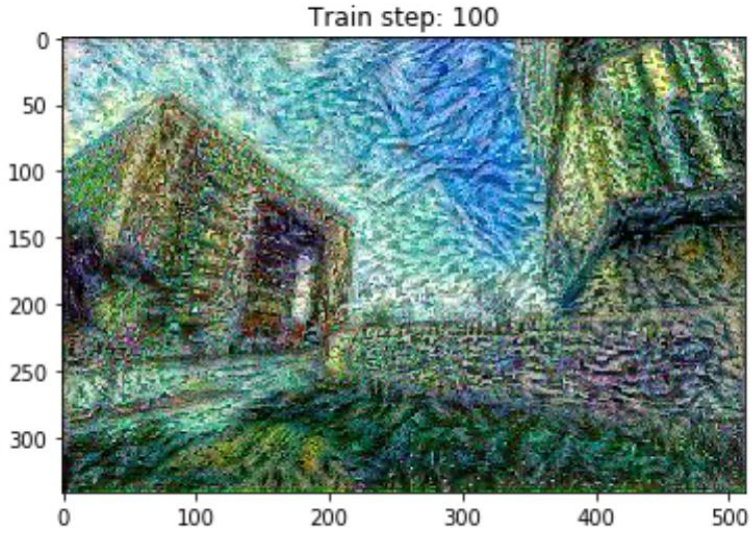


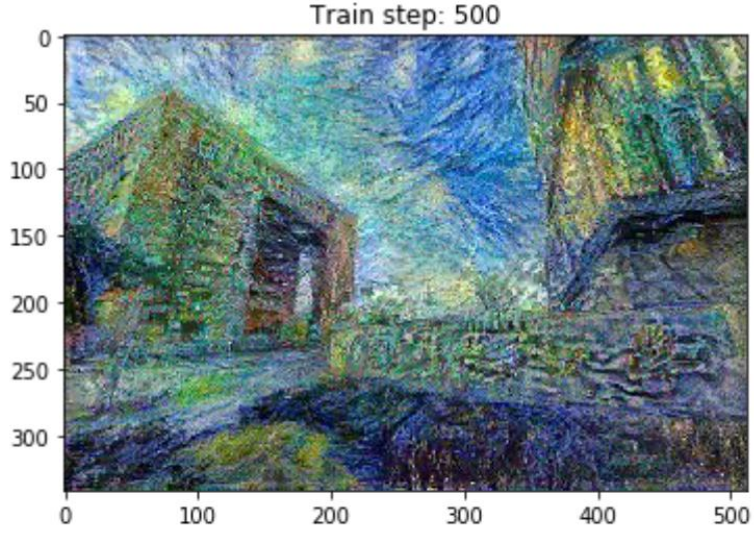
Style Image

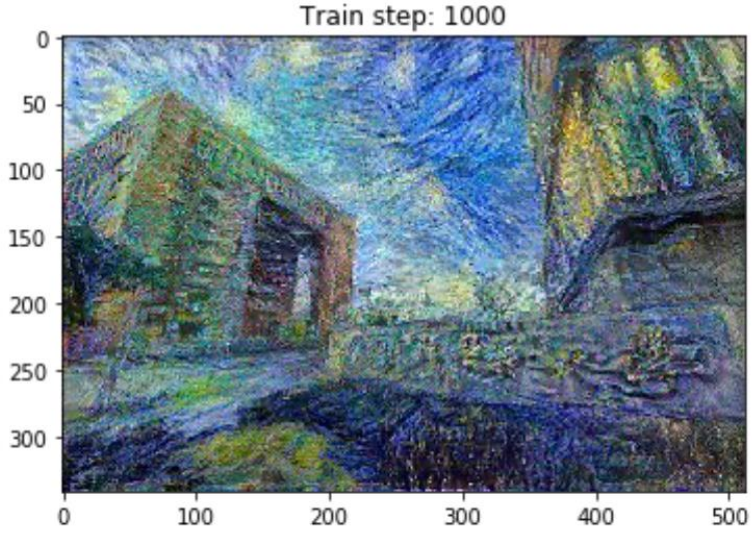


A Neural Algorithm of Artistic Style









A Neural Algorithm of Artistic Style

- Decrease these using an explicit regularization term on the high frequency components of the image. In style transfer, this is often called the total variation loss.

$$V(y) = \sum_{i,j} \sqrt{|y_{i+1,j} - y_{i,j}|^2 - |y_{i,j+1} - y_{i,j}|^2}$$

- In practice, to speed up the computation, we implement the following version instead:

$$V(y) = \sum_{i,j} |y_{i+1,j} - y_{i,j}| + |y_{i,j+1} - y_{i,j}|$$

A Neural Algorithm of Artistic Style

- w/ total_loss



w/o total_loss



Outline

- Visualization
- Neural Style Transfer
  - A Neural Algorithm of Artistic Style
  - AdaIN (Adaptive Instance Normalization)
- Save and Load models (Checkpoints)

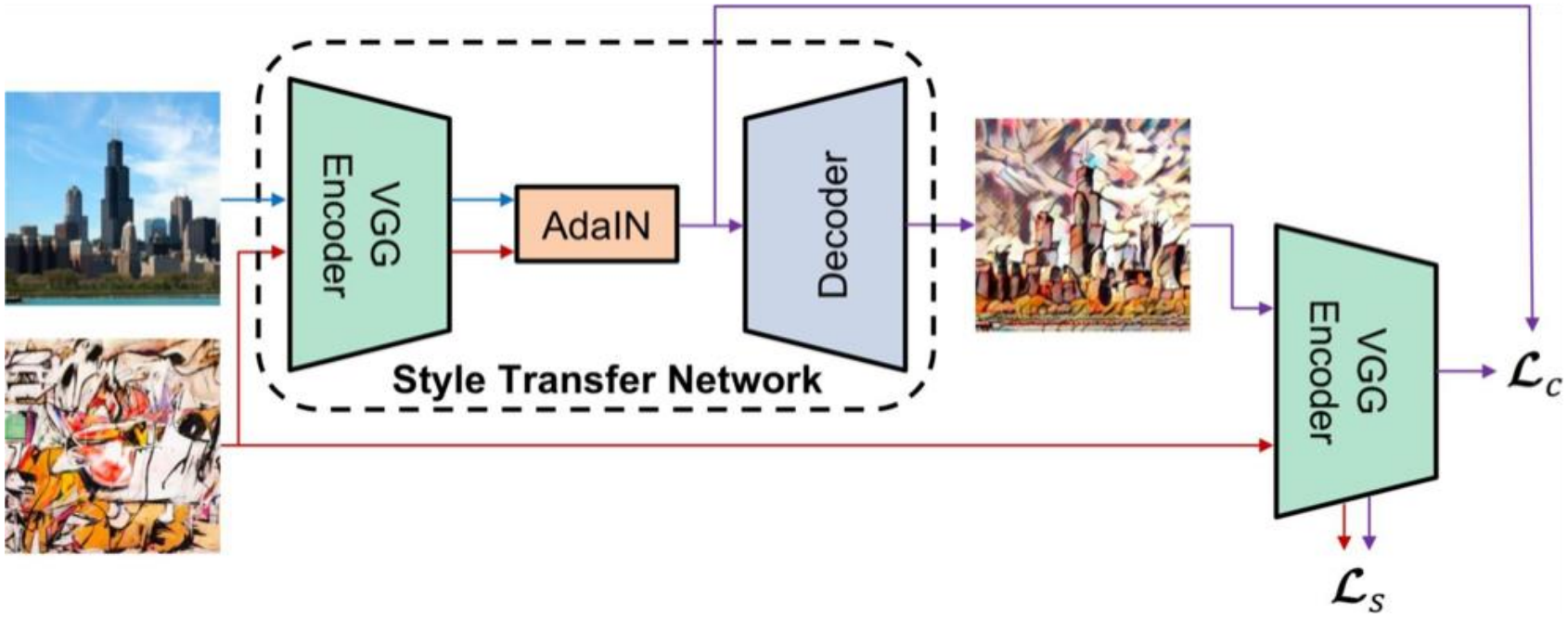
AdaIN (Adaptive Instance Normalization)

- The method we mentioned above requires a slow iterative optimization process, which limits its practical application.
- Xun Huang and Serge Belongie from Cornell University propose another framework, which enables arbitrary style transfer in real-time, known as "Arbitrary Style Transfer in Real-time with Adaptive Instance Normalization".

AdaIN (Adaptive Instance Normalization)

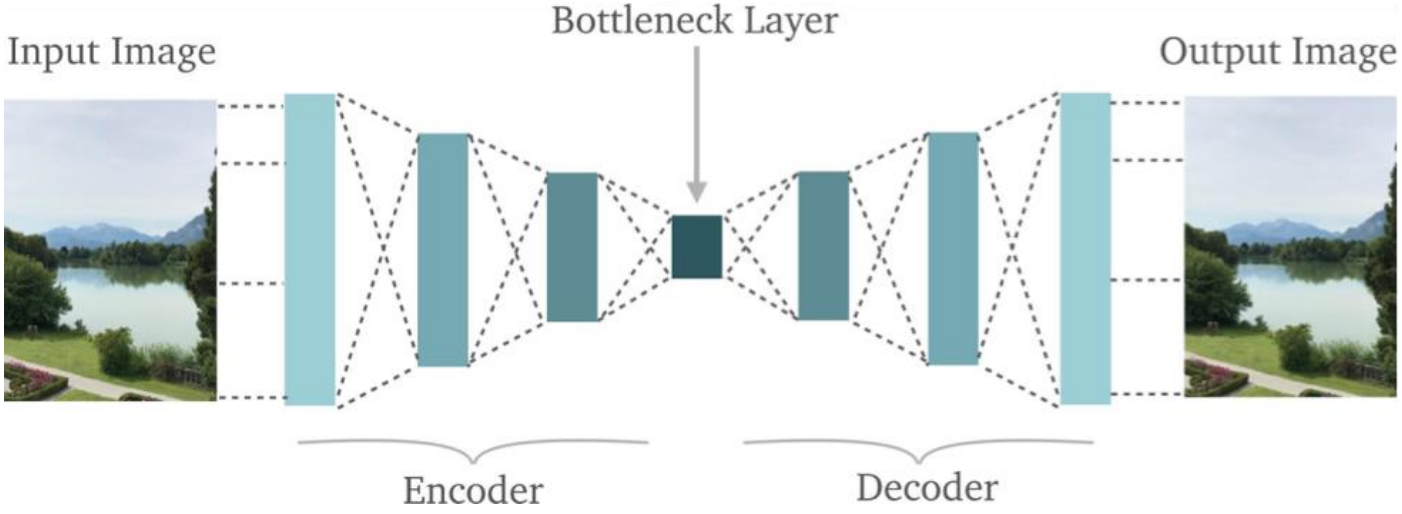
- At the heart of this method is a novel Adaptive Instance Normalization (AdaIN) layer aligning the mean and variance of the content features with those of the style features.
- Instance normalization performs style normalization by normalizing feature statistics, which have been found to carry the style information of an image in the earlier works.

AdaIN (Adaptive Instance Normalization)



AdaIN (Adaptive Instance Normalization)

- What are encoder and decoder?
  - Take autoencoder as an example:



AdaIN (Adaptive Instance Normalization)

- AdaIN receives a content input and style input, and simply aligns the channel-wise mean and variance of to match those of.
- It is worth knowing that unlike BN(Batch Normalization), IN(Instance Normalization) or CIN(Conditional Instance Normalization), AdaIN has no learnable affine parameters. Instead, it adaptively computes the affine parameters from the style inputs:

$$\text{AdaIN}(x, y) = \sigma(y)\left(\frac{x - \mu(x)}{\sigma(x)}\right) + \mu(y)$$

